

ZYZYX AUTHORITY SYSTEM™

Authority at Execution

Understanding How Automated Systems Actually Make Decisions

Systems don't fail at the start — they fail while they are running.

Document Code	AVE — 003
Document Type	Public System Behaviour Clarification
System Layer	1.5 — Understanding Layer
Classification	Public
Version	1.0 — Public Doctrine Edition
Publisher	Zyzyx.Tech — 2026
Series	Doctrine & Authority

DOCUMENT CLASSIFICATION

Document Title	Authority at Execution
Document Type	Public System Behaviour Clarification
System Layer	1.5 — Understanding Layer
Role	Explains, in clear everyday language, how automated systems behave during operation and why authority must be validated at the moment of action.
Core Function	Establishes why 'time of execution' and 'valid authority' are necessary conditions for fairness, accountability, and control.
Depends On	Public Automation Governance Framework (PAGF-002)
Enables	Human Authority Failure & Continuity Framework (HAFCF), Human-Bound Artificial Intelligence (HBAI), Execution-Layer Control Infrastructure (ELCI)
System Position	Bridges public understanding, system behaviour, and failure logic.
End Condition	The reader understands why systems must be governed at the moment they act, not just when they begin.

SYSTEM INTEGRATION STATEMENT

This document forms part of the Zyzyx Authority System, a unified governance architecture consisting of interdependent doctrine, validation models, infrastructure layers, and enforcement mechanisms.

This document must be read in conjunction with:

LTH	Letter to Humanity — A Citizen's Foundation for Automated Society
PAGF	Public Automation Governance Framework
HAFCF	Human Authority Failure & Continuity Framework
HBAI	Human-Bound Artificial Intelligence
ELCI	Execution-Layer Control Infrastructure
AFAF	Automation Failure Analysis Framework
BAF	Business Adoption Framework
EMF	Enforcement & Mandate Framework
RSGD	Robotic Systems Governance Doctrine (HBR)
FLAG	Foundational Laws of AI Governance

These documents do not operate independently. They form a single system.

Governance begins with understanding. It becomes enforceable only when aligned across all layers.

NOTICE

This document explains how modern automated systems behave in real-world conditions.

It avoids unnecessary technical language and focuses on practical understanding.

It does not describe how systems are built. It explains how they act — and why that matters.

This paper explains behavioural conditions, not implementation methods. 'Authority at execution' is a governance principle, not an implementation disclosure. No APIs, state machines, authentication protocols, or biometric workflows are described.

SECTION I

Why This Paper Exists

Most people believe that once a system is approved, logged in, or set up, it will continue to behave correctly.

That belief feels logical because it reflects how simpler systems used to work.

In the past, actions were direct. A person initiated something, it happened, and then it stopped. Responsibility was clear and immediate.

Modern systems are different.

They do not stop. They continue.

They operate in the background, processing information, triggering actions, and influencing outcomes continuously.

A system might:

- continue acting after the user has left
- continue making decisions based on earlier conditions
- continue applying rules without rechecking context
- continue influencing outcomes long after the original input

This creates a structural problem.

A system can begin correctly and still produce incorrect results later.

The longer it runs without rechecking, the greater the risk becomes.

This is not a rare issue. It is how most modern failures occur.

This paper exists to explain that behaviour clearly, and to show why control must exist at the moment an action happens.

SECTION II

What "Time of Execution" Actually Means

"Time of execution" is the exact moment an action takes place.

- Not when a system is started
- Not when permission is granted
- Not when a user logs in

It is the moment something actually happens. This is the point where consequences become real.

Examples include:

- the moment money is transferred
- the moment a decision is made about a person
- the moment access is granted or removed
- the moment content is blocked or allowed
- the moment a machine moves or performs a task

These are not preparation steps. They are outcome moments.

Everything before them leads up to them. Everything after them reacts to them.

If something is wrong, it is wrong at that moment.

That is why control must exist at that exact point. If control only exists before or after, then it does not exist where it matters.

SECTION III

What "Valid Authority" Actually Means

"Valid authority" means there is a real, correct, and current reason for an action to happen.

It is not enough that something was allowed earlier. It must still be allowed now.

Authority must meet four conditions:

It must be real	It must come from a person or institution that is accountable.
It must be current	It cannot rely on outdated information or past approval.
It must match the situation	The conditions at the moment must still justify the action.
It must be confirmed at the moment of action	It cannot be assumed or carried forward.

If any of these conditions fail, authority is no longer valid.

A system can appear correct while acting without valid authority. That is where hidden failure begins.

SECTION IV

Permission Is Not Authority

Most systems today rely on permission.

Permission means something is allowed. For example:

- a user is allowed to access a system
- a system is allowed to perform a function
- a process is allowed to run

Permission is typically granted once. After that, systems assume it remains valid.

Authority is different. Authority asks:

- should this still happen now?
- are the conditions still correct?
- is this still appropriate and safe?

Permission	Fixed. Granted once. Assumed to remain valid.
Authority	Continuous. Must be checked at every action.

Permission allows action. Authority justifies action. Without authority, permission becomes risk.

SECTION V

Why Systems Continue Acting Without Rechecking

Modern systems are designed to:

- operate continuously
- respond quickly
- act without interruption
- scale across large environments

These features create efficiency. But they also create a blind spot.

Systems are not naturally designed to stop and ask if they should continue. They are designed to keep going.

This means:

- actions continue after conditions change
- decisions repeat without reassessment
- processes carry forward outdated assumptions

This is not a malfunction.

It is expected behaviour based on how systems are built.

Without an explicit check at the moment of execution, systems will continue acting regardless of whether they should.

SECTION VI

Real-World Examples of the Problem

Bank Transfer

A user logs in and gains access.

Later, a transfer occurs. The system processes it because the login is still valid.

But if control of the session has changed, the action should not occur.

The system used permission. It did not confirm authority.

Loan Decision

A system evaluates an application. It uses available data.

If the data is incomplete or incorrect, the decision may still be made.

The system followed its rules. But it did not confirm whether the outcome was still valid at the moment of execution.

Social Media Moderation

A system removes content automatically. It matches a pattern. But context matters.

Without checking context at the moment of removal, the system may act incorrectly.

Workplace Access

Access is granted once. Conditions change. Access continues.

The system relies on past permission rather than current authority.

Physical Systems

A machine is operating safely. Conditions change.

If it continues without rechecking, it becomes unsafe.

The system is following earlier conditions, not current reality.

SECTION VII

The Core Question Every System Must Answer

Every action must answer one question:

"Is this still correct right now?"

Not earlier. Not assumed. Right now.

This question sounds simple. In practice, it is the question that most systems are never designed to ask. They are designed to proceed. Proceeding is efficient. Checking is friction. And so the check is removed — often gradually, often without a deliberate decision to remove it, but removed nonetheless.

If that question is not asked, the system is operating on assumption.

Assumption is not control.

A system that proceeds without asking whether it should is not a governed system. It is a system that has been permitted to act — and permission, once granted, is not the same as ongoing authority.

SECTION VIII

Why Problems Occur During Operation, Not at the Start

Systems are often tested at the beginning.

They are validated when they are set up. They are checked when they are deployed.

But most problems do not occur at those points. They occur later.

They occur because:

- conditions change
- data becomes outdated
- behaviour evolves

- interactions become more complex

The longer a system operates without revalidation, the further it can move from correct behaviour. This is why failure happens during execution, not at the start.

SECTION IX

Hallucination — When Systems Are Confident but Wrong

Some systems can produce outputs that appear accurate, detailed, and confident. But those outputs can be incorrect.

This is called hallucination.

It means:

- the system presents something as true
- even when it is not

This is dangerous because:

- the output looks trustworthy
- the system may not recognise the error
- people may act on incorrect information

If an action follows a hallucinated output without checking authority at execution, the result can be harmful while appearing valid.

SECTION X

Drift — When Systems Slowly Move Away from Accuracy

Drift is gradual change over time.

A system may start correctly. But over time:

- its outputs shift
- its accuracy changes
- its behaviour moves away from original expectations

Drift is difficult to detect because it happens slowly. By the time it becomes obvious, many actions may already have been affected.

Without checking authority at execution, drift allows systems to continue acting even when they are no longer correct.

SECTION XI

Misalignment — When Systems Follow Rules but Get It Wrong

A system can follow its rules perfectly and still produce the wrong outcome.

This happens when:

- the rules do not match reality
- context is missing
- priorities are incorrect

The system is technically correct. But practically wrong.

Without checking authority at the moment of action, the system continues anyway.

SECTION XII

Why These Problems Require Human-Bound Systems

Hallucination, drift, and misalignment show that systems cannot be left to operate independently without oversight.

They can:

- appear correct when they are wrong
- change without being noticed
- apply rules without understanding context

This leads to a clear requirement: systems must remain connected to human authority.

This is the basis of Human-Bound AI.

It means:

Actions are linked to real responsibility	Every action traces back to an accountable human or institution.
Authority is confirmed before actions occur	The system does not proceed unless current conditions justify it.
Systems do not act purely on assumption	Validation happens at execution, not only at setup.
Humans remain accountable where consequences exist	Accountability is not delegated to the system.

This does not prevent automation.

It ensures automation remains aligned with reality.

SECTION XIII

What Proper Control Looks Like

A properly governed system:

- checks conditions before every action
- confirms authority at the moment of execution
- stops when conditions are no longer valid
- keeps responsibility clear and traceable

The difference between these two descriptions is not a matter of speed or efficiency. A governed system can still act quickly. What it cannot do is act without authority. The condition that must be satisfied before execution is not bureaucratic overhead — it is the mechanism that makes accountability possible.

Actions are not automatic. They are conditional.

The system does not assume it is correct. It proves it is correct before acting.

SECTION XIV

Why This Matters

This affects everyday life:

Financial Decisions	Approvals, transfers, and credit assessments.
Access to Services	Healthcare, welfare, and public administration.
Employment Opportunities	Hiring, screening, and workforce decisions.
Information Access	Content, platforms, and communication systems.
Personal Rights	Legal status, identity, and civic participation.

If systems act without valid authority at execution, they can affect outcomes without accountability. That is not a technical issue. It is a societal issue.

SECTION XV

Closing Doctrine

Authority does not exist at the start of a process. It must exist at the moment something happens.

If a system cannot show that an action was:

- correct
- appropriate
- properly authorised

at the exact moment it occurred — then it is not controlled.

FINAL STATEMENT

If it is not authorised at the moment it happens, it should not happen at all.

© Zyzyx.Tech — All Rights Reserved. Public Governance Doctrine Edition.